ÉCOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT TECHNIQUE
ENSET Mohammedia

ADRIA Banking Platform

Rapport de l'épreuve technique de sélection

Architecture microservices, intelligence artificielle et développement full-stack

Candi
Format
Établissem
Épre
D

Dépôt du code :
https://github.com/Marouanemounir/mounir-marouane-test-adria-pfe-27-

Rapport de conception, d'implémentation et de vérification — avec preuves disponibles.

Résumé

Ce rapport présente la conception et la réalisation d'une plateforme bancaire distribuée répondant à l'épreuve ADRIA ENSET PFE 2027. Deux microservices métier assurent la gestion des comptes et des transactions. Spring Cloud fournit la découverte, la configuration et le routage ; OpenFeign et Resilience4j structurent les appels synchrones ; Kafka prend en charge la diffusion d'événements. Des interfaces REST, GraphQL et MCP permettent l'exposition de fonctionnalités, avec un service d'assistant basé sur Spring AI et un client web Angular. Un volet indépendant en Python traite la classification des fraudes. Le document distingue systématiquement le code observé, les captures de démonstration et les scénarios qui ne sont pas prouvés par les éléments transmis.

Mots clés : Spring Boot, Spring Cloud, Java 21, Eureka, OpenFeign, Resilience4j, Kafka, GraphQL, MCP, Spring AI, Angular, scikit-learn, CI/CD.

Table des matières

Résumé **i**

1 Contexte, objectifs et périmètre **1**
1.1 Problématique métier . . . 1
1.2 Exigences de l'épreuve . . . 1
1.3 Périmètre et statut des preuves . . . 1

2 Partie A — Préparation et gestion du projet **2**
2.1 Organisation Maven multimodule . . . 2
2.2 Compilation et versionnement . . . 3

3 Partie B — Conception et architecture **4**
3.1 Architecture cible . . . 4
3.2 Conception orientée objet . . . 4

4 Partie C — Microservices métier **6**
4.1 Account Service . . . 6
4.2 Transaction Service . . . 6
4.3 Communication, tolérance et cohérence . . . 6
4.4 Interfaces REST, GraphQL et MCP . . . 6

5 Partie D — Services techniques Spring Cloud **7**
5.1 Découverte Eureka . . . 7
5.2 Configuration centralisée . . . 7
5.3 Passerelle d'API . . . 8

6 Partie E — Assistant bancaire intelligent **9**
6.1 Approche agentique . . . 9
6.2 Interface de conversation . . . 9

7 Partie F — Communication événementielle Kafka **10**

8 Partie G — Client web Angular **11**
8.1 Architecture frontend . . . 11
8.2 Tableau de bord . . . 11
8.3 Virements . . . 12

9 Partie H — Détection de fraude avec Python **13**
9.1 Objectif et données . . . 13
9.2 Méthode . . . 13
9.3 Interprétation et limites . . . 13

10 Partie I — DevOps et industrialisation **14**
10.1 Conteneurisation 14
10.2 Chaîne CI/CD demandée 14
10.3 Mesures d'exploitation 14

11 Synthèse, résultats et perspectives **15**
11.1 Tableau de couverture 15
11.2 Bilan 15

Annexe — Reproduction rapide **16**

1. Contexte, objectifs et périmètre

1.1 Problématique métier

La banque souhaite consulter les comptes, enregistrer les transactions, mettre à jour les soldes, effectuer des virements et émettre des événements métier. Un assistant conversationnel doit faciliter la consultation des données. Le sujet prévoit aussi une expérimentation indépendante de détection de fraude sur données transactionnelles.

1.2 Exigences de l'épreuve

Le sujet, d'une durée indicative de trois heures, se décompose en neuf parties : A — consignes de livraison ; B — conception UML ; C — services métier REST/GraphQL/MCP et résilience ; D — Gateway/Eureka/Config ; E — agent IA ; F — Kafka ; G — client web ou mobile ; H — apprentissage automatique Python ; I — conteneurisation et CI/CD. Les règles bancaires sont : montant positif, plafond configuré à 1000, compte existant et actif, devise cohérente, consultation du solde, mise à jour et transfert.

1.3 Périmètre et statut des preuves

Les sources disponibles comprennent les fichiers du dépôt GitHub consulté et les captures fournies par le candidat. La capture Maven prouve une compilation réussie des modules Java, mais ne constitue pas une preuve des tests métier. Les captures Angular montrent le rendu des pages, avec des listes vides lors de la prise de vue ; elles ne prouvent pas à elles seules les opérations bancaires. Une capture Eureka atteste l'enregistrement des quatre applications visibles. Les fonctions LLM/MCP, publication Kafka et métriques ML sont documentées à partir du code ou des artefacts présents, mais leur exécution n'est pas démontrée par une capture dédiée dans les pièces fournies.

2. Partie A — Préparation et gestion du projet

2.1 Organisation Maven multimodule

Le projet utilise Java 21, Spring Boot 3.5.16, Spring Cloud 2025.0.3 et un parent Maven. Les modules présents sont : `account-service`, `transaction-service`, `discovery-service`, `config-service`, `gateway-service` et `ai-agent-service`. Les répertoires `banking-web`, `fraud-detection`, `scripts` et `docs` complètent les livrables.

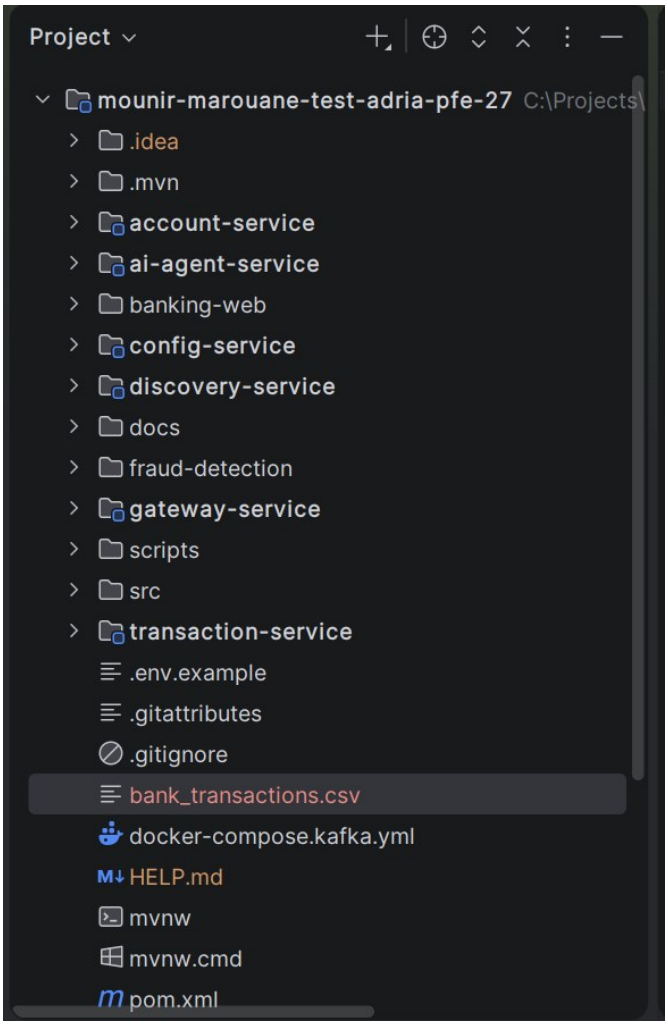


FIGURE 2.1 – Arborescence du projet dans IntelliJ IDEA (capture fournie).

2.2 Compilation et versionnement

La compilation Maven produite dans les captures se termine par `BUILD SUCCESS` ; le reactor présente tous les modules Java en succès. Les commits GitHub successifs matérialisent les étapes métiers, Cloud, Kafka et IA. La remise finale doit également inclure le repository et le rapport PDF dans Classroom, conformément au sujet.

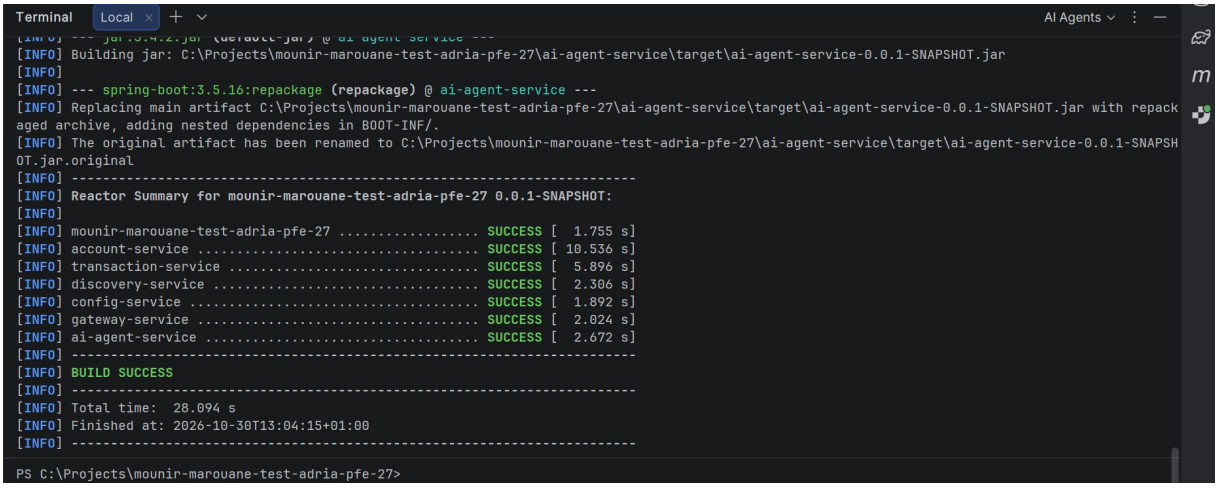


FIGURE 2.2 – Compilation Maven du parent et des six modules : résultat `BUILD SUCCESS`.

3. Partie B — Conception et architecture

3.1 Architecture cible

L'application sépare les clients, les composants techniques, les services métiers, l'assistant IA, les échanges Kafka et les bases de données. Le client Angular utilise la Gateway ; Transaction Service consulte Account Service au moyen d'OpenFeign. Les services déclarent leurs instances dans Eureka et récupèrent les propriétés partagées dans Config Server.

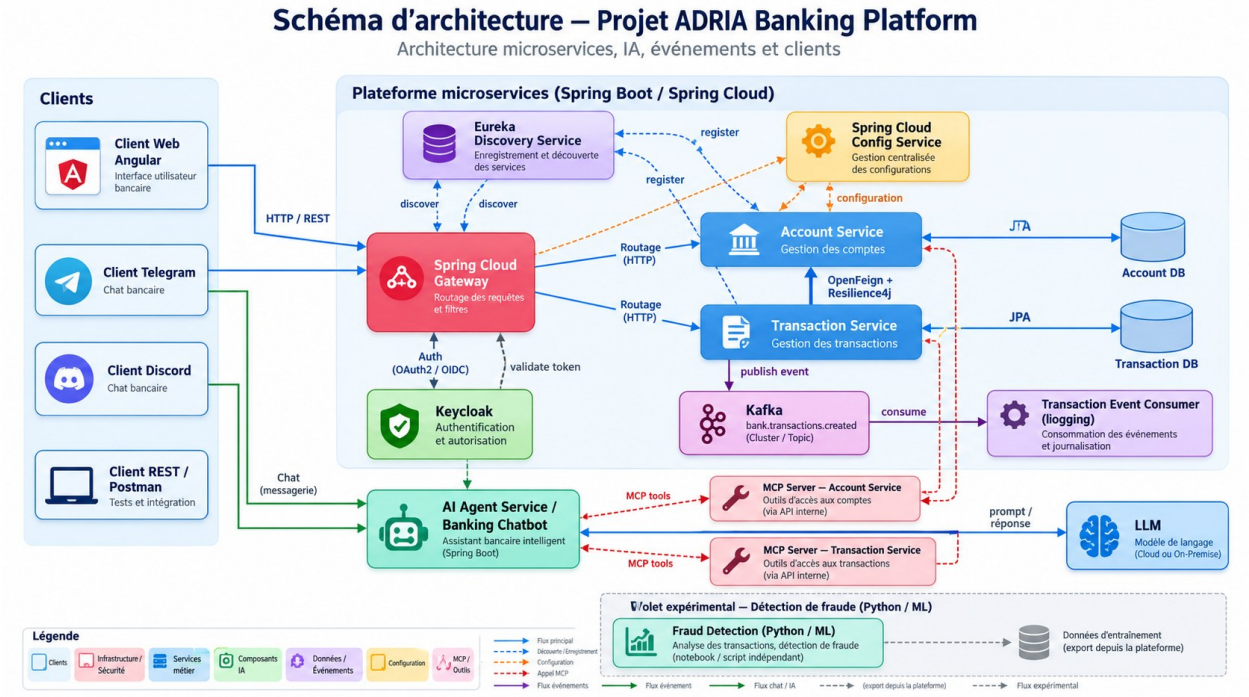


FIGURE 3.1 – Schéma d'architecture global fourni. Il illustre une architecture cible ; certains éléments, notamment Keycloak et Telegram/Discord, ne sont pas établis comme implémentés.

3.2 Conception orientée objet

L'entité `Account` porte `id`, `customerReference`, `status`, `currency` et `balance` ; `Transaction` comporte `id`, `accountId`, `amount`, `currency`, `merchantCategory`, `country` et `createdAt`. Les contrôleurs et outils dépendent de services partagés et les repositories encapsulent les accès aux données.

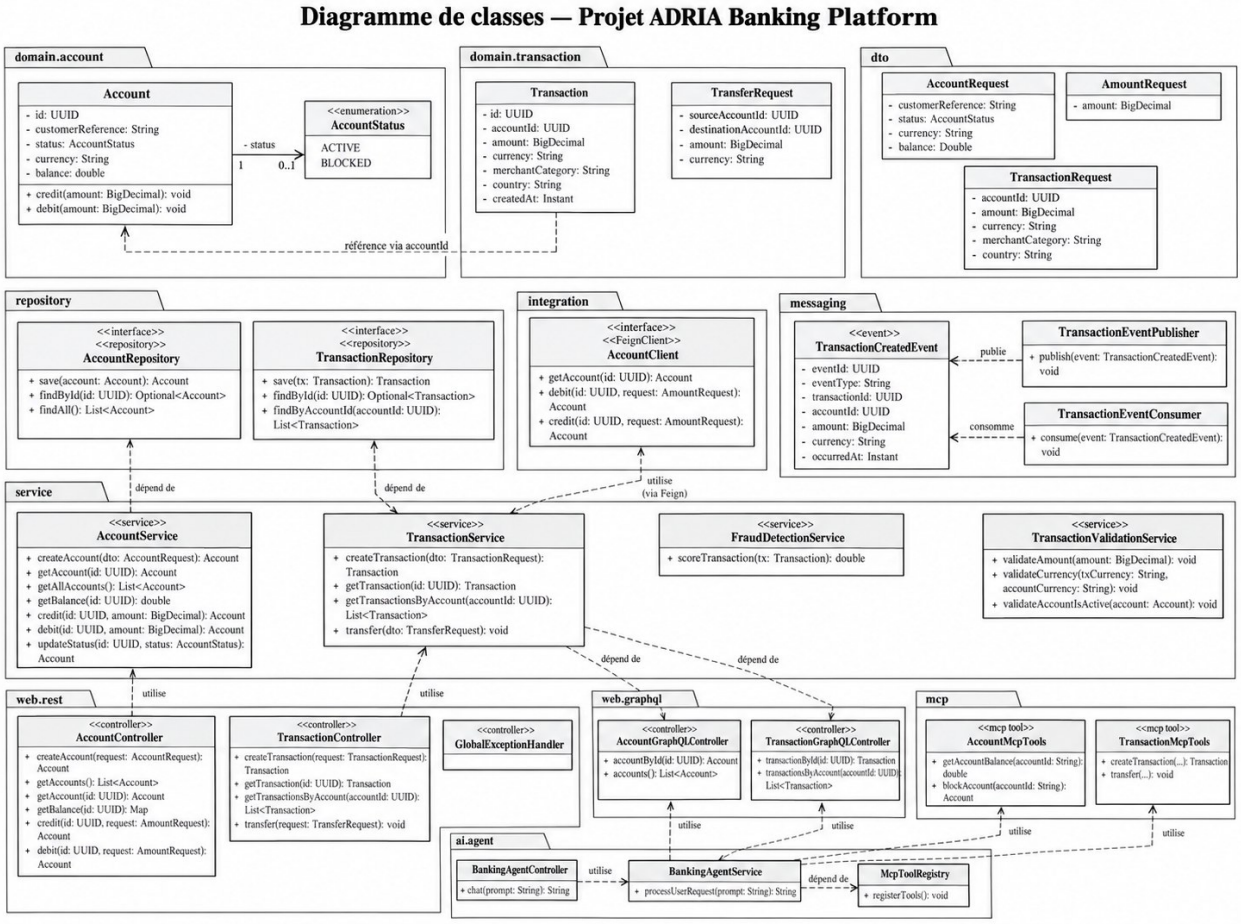


FIGURE 3.2 – Diagramme de classes UML fourni : modèle conceptuel élargi. Il ne doit pas être confondu avec un relevé exhaustif des classes effectivement déployées.

4. Partie C — Microservices métier

4.1 Account Service

`account-service` expose une API REST avec création, consultation, solde, crédit, débit et modification du statut. Il comporte des couches `domain`, `repository`, `service` et `web.rest`, une base H2 et des tests d'intégration. L'entité `Account` distingue les statuts `ACTIVE` et `BLOCKED`. La séparation entre API et service évite la duplication des règles métier.

4.2 Transaction Service

`transaction-service` gère les transactions et virements, avec modèle JPA, repositories, DTO, gestion centralisée des erreurs et identifiants d'opérations. Le code comprend `AccountClient`, `AccountGateway`, `TransactionService`, `TransactionOperation` et `OperationStore`. Le montant maximal est une propriété configurée : `bank.transaction.max-amount=1000`. Les vérifications incluent existence/état du compte, cohérence de devise et solde suffisant.

4.3 Communication, tolérance et cohérence

OpenFeign encapsule les appels HTTP interservices et Resilience4j gère les indisponibilités. L'idempotence est cruciale pour éviter les doubles débits. Une transaction JPA locale ne garantit pas l'atomicité de deux bases distinctes : les opérations distribuées nécessitent identification d'opération, suivi d'état et traitement prudent des échecs incertains. Cette limite doit être rappelée avant un usage bancaire réel.

4.4 Interfaces REST, GraphQL et MCP

Les schémas GraphQL et classes de résolution existent dans les deux services ; des classes `AccountMcpTools` et `TransactionMcpTools` exposent les outils MCP. Les trois interfaces sont conçues pour réutiliser les services métier plutôt que d'implémenter leurs propres règles.

5. Partie D — Services techniques Spring Cloud

5.1 Découverte Eureka

Le serveur Eureka fonctionne sur le port 8761. La capture fournie affiche `ACCOUNT-SERVICE`, `AI-AGENT-SERVICE`, `GATEWAY-SERVICE` et `TRANSACTION-SERVICE` dans l'état `UP`.

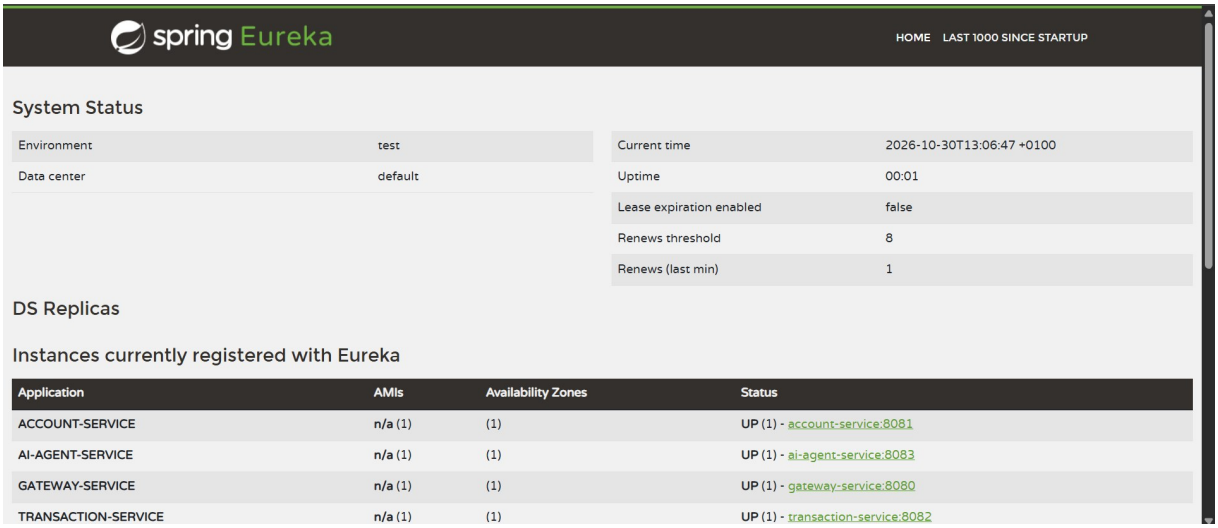


FIGURE 5.1 – Console Eureka : services métier, Gateway et agent IA enregistrés en état UP.

5.2 Configuration centralisée

Config Server, sur le port 8888, fournit une configuration en mode natif, dont la propriété de plafond bancaire. La capture JSON montre explicitement la valeur `1000` dans `classpath:/config-repo/applicati`

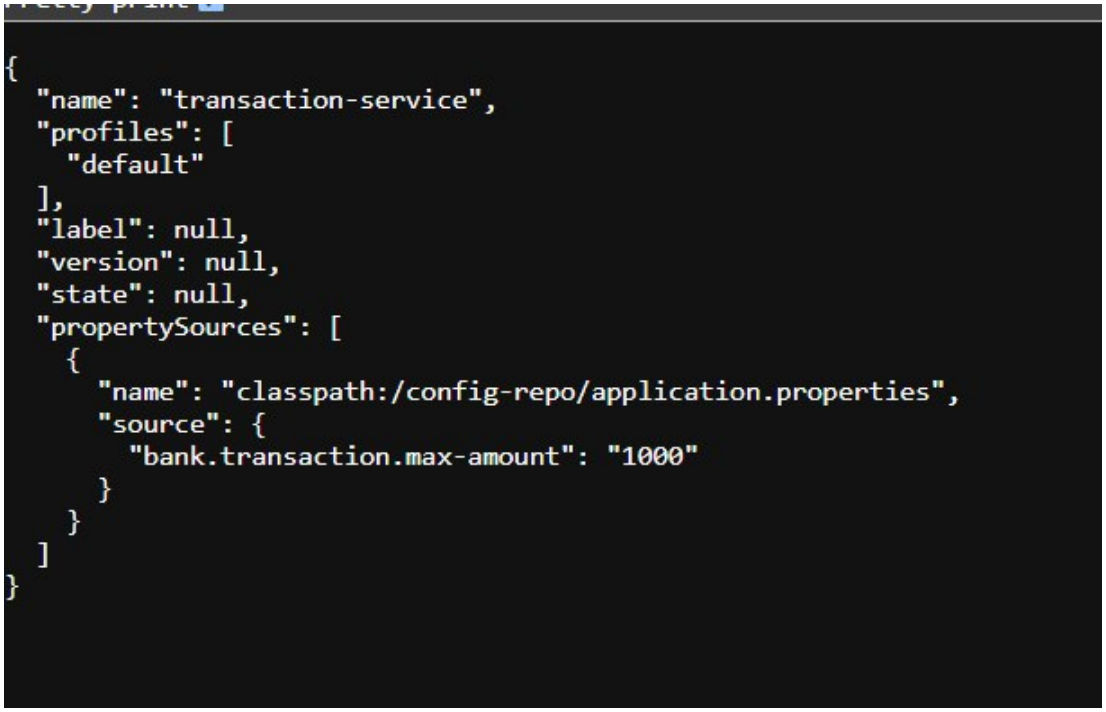


FIGURE 5.2 – Réponse de Config Server pour `transaction-service/default`, incluant le plafond de transaction.

5.3 Passerelle d'API

Spring Cloud Gateway assure l'entrée principale des API bancaires via le port 8080 et le routage vers les services découverts. Le code de configuration existe dans le module `gateway-service` ; une capture spécifique de requête traversant la passerelle n'a pas été fournie.

6. Partie E — Assistant bancaire intelligent

6.1 Approche agentique

Le module `ai-agent-service` contient une configuration `ChatClient`, un client MCP, `BankingAgentService` un contrôleur de chat et un filtre d'accès. Le principe est la sélection d'outils par le modèle de langage pour lire les données bancaires réelles des microservices. L'exécution complète exige un fournisseur LLM configuré et des autorisations appropriées.

6.2 Interface de conversation

Le frontend Angular présente une page de chat dédiée, avec zone de saisie, propositions de questions et indications de connexion à l'agent bancaire. La capture atteste l'interface mais ne présente pas de réponse LLM ni de trace d'appel MCP.

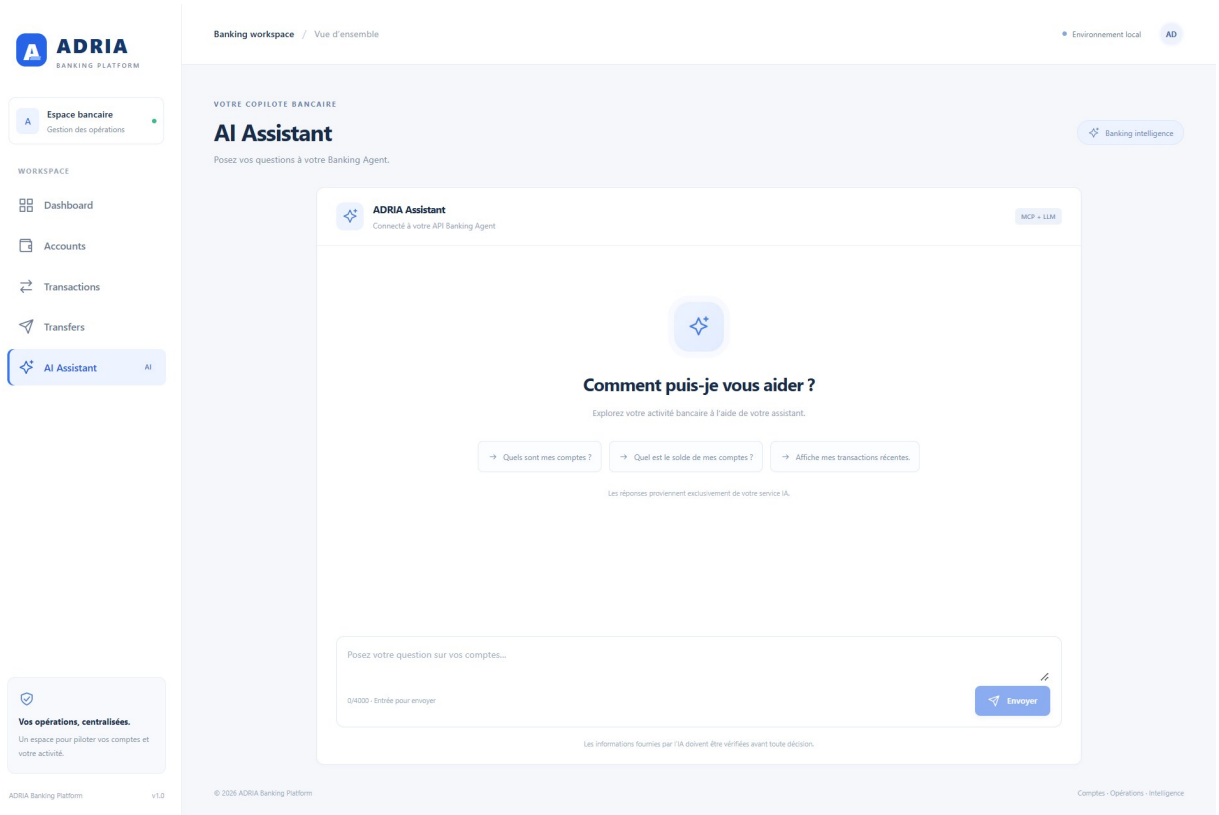


FIGURE 6.1 – Interface de l'assistant bancaire Angular, état initial sans conversation démontrée.

7. Partie F — Communication événementielle Kafka

Le module Transaction Service contient `TransactionCreatedEvent`, `KafkaConfiguration`, `TransactionEventPublisher` et `TransactionEventConsumer`. Le topic imposé est `bank.transactions.created` la clé attendue est `accountId`. Le consumer journalise les événements sans modifier les données bancaires. Un fichier `docker-compose.kafka.yml` configure un broker Kafka en mode KRaft. Le code est présent, mais aucune capture de message consommé ne figure dans les preuves remises.

```
{ "eventId": "UUID", "eventType": "TransactionCreated",
  "transactionId": "UUID", "accountId": "UUID",
  "amount": 250.00, "currency": "EUR",
  "occurredAt": "2026-10-30T10:30:00Z" }
```

8. Partie G — Client web Angular

8.1 Architecture frontend

Le répertoire `banking-web` contient un client Angular avec navigation, services d'API, modèles TypeScript, état applicatif et pages `Dashboard`, `Accounts`, `Transactions`, `Transfers` et `Assistant`. Le recours à un proxy local facilite l'intégration aux API Spring Boot.

8.2 Tableau de bord

La page dashboard regroupe un espace de synthèse, des compteurs, les soldes par devise et les transactions récentes. La capture montre zéro compte et zéro transaction à l'instant de la prise de vue : elle atteste le rendu, pas le chargement effectif de données non vides.

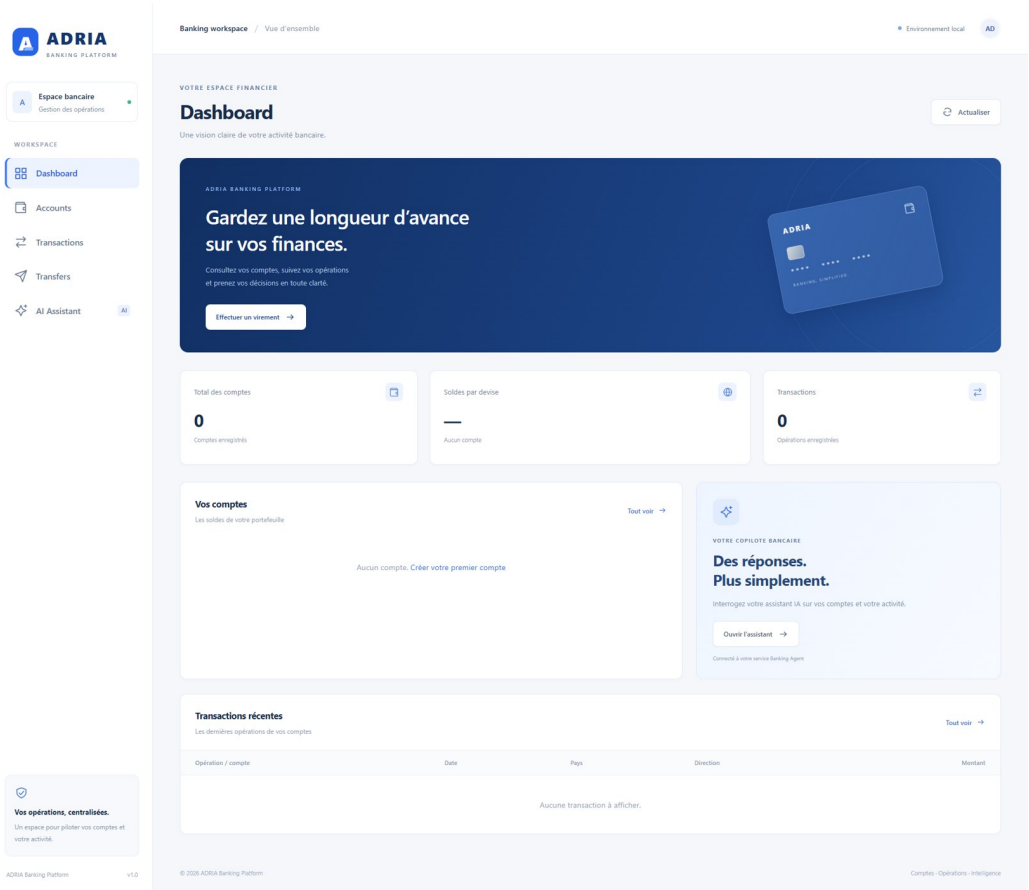


FIGURE 8.1 – Tableau de bord Angular affiché dans le navigateur, état initial sans données.

8.3 Virements

L'écran des virements présente la sélection du compte débiteur et du compte créditeur, le montant et la devise. Des consignes rappellent le plafond indicatif et le rôle du backend dans la validation finale.

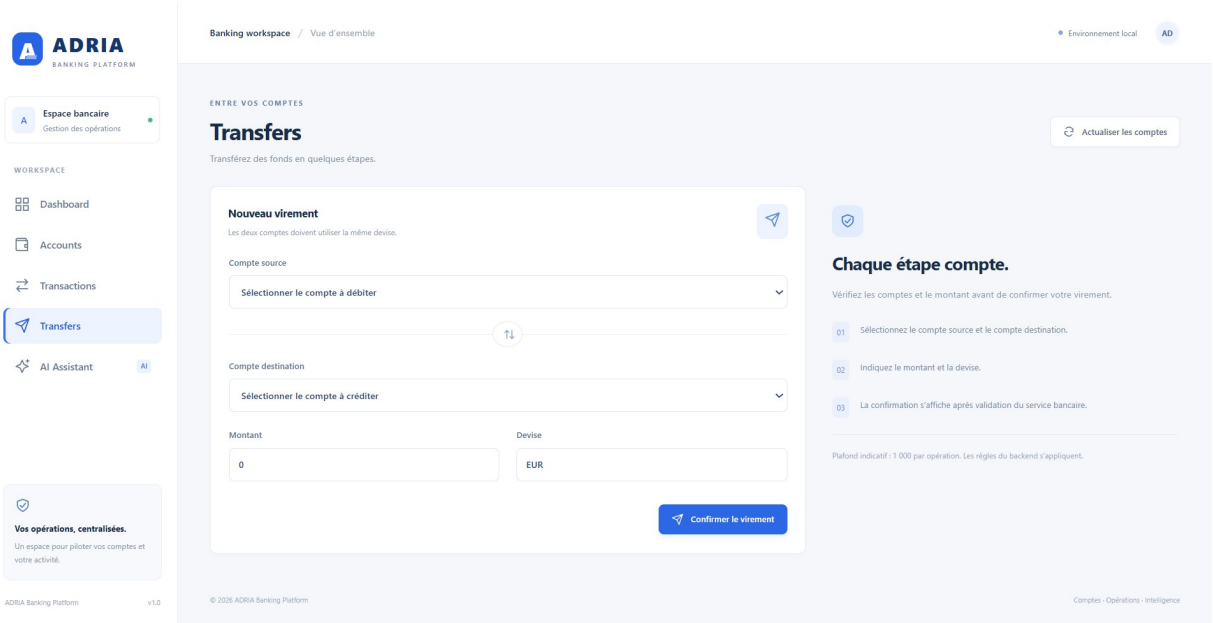


FIGURE 8.2 – Formulaire Angular de virement. La capture ne démontre pas l'exécution d'un virement.

9. Partie H — Détection de fraude avec Python

9.1 Objectif et données

La classification binaire distingue `is_fraud=0` et `is_fraud=1`. Les variables du sujet sont : `amount`, `hour`, `country`, `merchant_category`, `transactions_last_24h`, `average_amount_last_7d`, avec exclusion des identifiants. Le répertoire `fraud-detection` comporte le script `fraud_detection.py`, un CSV, un modèle sérialisé et des résultats enregistrés.

9.2 Méthode

La procédure exigée prévoit analyse des valeurs manquantes et du déséquilibre, séparation entraînement/test stratifiée, pipeline d'imputation/encodage/standardisation, puis classification simple. Le prétraitement doit être ajusté sur l'entraînement exclusivement. Les indicateurs utiles sont précision, rappel et F1 de la classe fraude, matrice de confusion et average precision.

9.3 Interprétation et limites

1. **Accuracy insuffisante :** si les fraudes sont rares, prédire systématiquement la classe majoritaire peut donner une accuracy élevée tout en manquant toutes les fraudes.
2. **Faux positifs et faux négatifs :** améliorer la détection peut accroître le nombre de transactions légitimes signalées ; le compromis dépend des coûts métier.
3. **Seuil de décision :** le diminuer tend à accroître le rappel, mais augmente généralement les faux positifs.
4. **Validation temporelle :** elle reflète mieux une prédiction sur transactions futures et réduit les fuites provenant d'informations postérieures.

Des figures et métriques sont présentes dans le dépôt (`results/metrics.json`, `confusion_matrix.png`, `precision_recall_curve.png`). Leur provenance, le type de dataset et les valeurs exactes sont à confirmer avant d'en faire une conclusion expérimentale ; aucune capture de leur exécution n'a été fournie ici.

10. Partie I — DevOps et industrialisation

10.1 Conteneurisation

Un Docker Compose dédié au broker Kafka est présent. La création de Dockerfiles pour *l'ensemble* des microservices et d'un Compose global reste un livrable distinct à vérifier et compléter : ils ne sont pas attestés dans les pièces examinées.

10.2 Chaîne CI/CD demandée

La chaîne cible comprend compilation Maven, tests, construction d'images, publication dans un registre et étape de déploiement. Le sujet autorise un pipeline schématique et ne demande aucun déploiement cloud réel. Aucun pipeline GitHub Actions complet n'a été établi parmi les fichiers examinés.

10.3 Mesures d'exploitation

— **Secrets :** clés API, jetons et mots de passe doivent être stockés en secrets ou variables d'environnement protégées ; la configuration non sensible relève de Config Server.
— **Probes :** startup pour l'initialisation, readiness pour la réception de trafic, liveness pour détecter un blocage.
— **Observabilité :** logs structurés, métriques Actuator/Micrometer et traces distribuées avec corrélation.
— **Rollback :** étiquetage des images et retour à la dernière version stable après validation.

11. Synthèse, résultats et perspectives

11.1 Tableau de couverture

Partie	Éléments disponibles	Niveau de preuve
A	Maven multimodule, dépôt, compilations	Capture BUILD SUCCESS
B	Schéma d'architecture et diagramme de classes	Diagrammes fournis (conceptuels)
C	Comptes, transactions, virements, Feign, résilience, GraphQL, MCP	Code présent ; parcours métier non capturé
D	Eureka, Config Server, Gateway	Eureka UP et valeur Config capturées
E	Agent Spring AI et interface chatbot	Code + interface ; réponse LLM non prouvée
F	Événement Kafka, producer/consumer, Compose Kafka	Code présent ; consommation non capturée
G	Client web Angular multi-pages	Captures de trois pages
H	Script ML, modèle et artefacts de métriques	Fichiers présents ; exécution non capturée
I	Docker Compose Kafka ; consignes CI/CD documentées	Pipeline complet non établi

11.2 Bilan

Le projet fournit une base technique large et cohérente pour une épreuve d'intégration microservices : découpage métier, communication synchrone et asynchrone, couche d'agent IA, visualisation Angular et expérimentation de détection de fraude. Les preuves disponibles démontrent la compilation Maven, l'enregistrement Eureka, la distribution du paramètre de plafond et l'existence d'une interface web soignée. Pour une validation complète, il conviendrait d'ajouter des captures d'opérations bancaires réelles, d'événement Kafka reçu, d'appel MCP fonctionnel, de métriques ML exécutées et de pipeline CI/CD. Les contraintes de sécurité, de cohérence distribuée et de persistance H2 doivent être renforcées avant tout usage en production.

Annexe — Reproduction rapide

```
# A la racine du projet sous Windows
.\mvnw.cmd -DskipTests clean package
# Broker Kafka
docker compose -f docker-compose.kafka.yml up -d
# Demarrer dans IntelliJ : discovery (8761), config (8888),
# account (8081), transaction (8082), gateway (8080),
# agent (8083) si configure.
# Client Angular
cd banking-web && npm install && npm start
# Volet ML
cd fraud-detection
python -m pip install -r requirements.txt
python fraud\_detection.py --data data/bank\_transactions.csv
```

Dépôt : https://github.com/Marouanemounir/mounir-marouane-test-adria-pfe-27-